

Capstone Completion Assessment — AI FDE CGT Patient-to-Batch Orchestration

Executive summary

The supplied archive, `AI_FDE_CGT_Patient_to_Batch_Orchestration.zip`, is **not a completed capstone submission**. It is a deliberately imperfect, runnable brownfield challenge environment intended to be analysed and modernised. The repository says this explicitly: the participant is expected to “discover before designing”, and the authoritative challenge brief defines ten participant deliverables ranging from forensic analysis and domain reconstruction through three workflow POCs, TEVV, security/governance, resilience, and a production-readiness roadmap (`README.md:3-13` ; `participant/CHALLENGE_BRIEF.md:5-19`).

The seed repository itself is in reasonably good condition as a training artefact: structural verification passes, SQLite integrity passes, all checksum entries validate, and the baseline test suite produces the documented result of **2 passed / 3 expected failures**. I independently reproduced those results. The three `xfail` tests are meaningful rather than incidental: they expose a false product-readiness rule, an obsolete cryogenic-excursion rule, and non-idempotent slot reservation (`tests/test_legacy_expected_failures.py:5-17`).

However, from a senior Full-Stack/FDE architecture perspective, the repository currently provides only a **small read-only FastAPI façade over a synthetic SQLite data estate**, plus diagnostics, challenge data, and deliberately defective legacy rules. There is **no frontend, no authentication or authorisation, no write/orchestration workflow, no CI/CD pipeline, no cloud/hosting definition, no Kubernetes/IaC, no current-system API contract, no migration framework, no production observability, no formal ADR set, and no participant-authored capstone deliverables**.

The strongest architectural observation is that the default “normal full-stack capstone” rubric should **not** be applied blindly. Generic CRUD would actually be the wrong design target for this domain. The repository’s own engineering guidance requires provenance, reversible changes, explicit state machines, idempotency, evidence-backed identity reconciliation and human authorisation for consequential actions (`AGENTS.md`). The correct product is therefore a **governed orchestration and decision-support system**, not an administrative CRUD application.

Three POCs should anchor the finished capstone:

Recommended POC	Purpose	Why it is the right proof point
Evidence-backed Journey & Identity Resolution	Reconcile CRM/clinical/patient/material/batch identifiers, construct an evidence-backed timeline, and visibly preserve contradictions	Directly addresses duplicate/conflicting identity, chain-of-identity and temporal anomalies

Recommended POC	Purpose	Why it is the right proof point
Governed Readiness & Exception Workbench	Present patient/batch readiness, blocking consent/QC/QA/site/logistics conditions, evidence and required human authority	Replaces the knowingly unsafe <code>patient_ready()</code> heuristic and demonstrates useful frontend/FDE value
Resilient Orchestration / Slot Reservation	Implement idempotent workflow commands, retry behaviour, compensation and outage simulation	Directly fixes the non-idempotent legacy slot service and demonstrates distributed-systems engineering

A practical target architecture is a thin responsive frontend, a versioned FastAPI API, canonical domain/application services, an immutable evidence/event layer, deterministic policy/state-machine components, explicit human-approval boundaries, and optional AI recommendation adapters that have **no autonomous release or clinical authority**.

The most important immediate blockers are:

Severity	Blocker
Critical	None of the ten participant capstone deliverables is fully delivered as a participant solution
Critical	Three knowingly defective domain behaviours remain executable baseline behaviour
High	No frontend or workflow POCs exist
High	No authentication, authorisation, audit trail or human-approval mechanism exists
High	No CI/CD or deployed demonstration exists
High	No integration/API/e2e/security/performance test suite exists; measured source coverage is only 59%
High	Package/version metadata is internally inconsistent despite documentation claiming normalisation
High	Dependency resolution is not reproducible: broad ranges, no lockfile, no hashes for dependencies
Medium	Docker image runs as root and copies the entire repository into the runtime image
Medium	SQLite schema has no declared PK/FK constraints and only three indexes
Medium	<code>.env.example</code> values are not actually consumed by application configuration
Medium	Current OpenAPI contracts do not describe the implemented FastAPI API
Medium	README says the dataset can be rebuilt, but supplied regeneration scripts do not actually regenerate it

Completion judgement: the starter repository is operationally usable as an assessment seed, but **the capstone itself should not be marked complete until the ten challenge-brief outputs, three demonstrable workflows, automated quality gates, a deployed demonstration, security/**

governance controls, and the completion documentation described below are delivered and reproducibly pass acceptance tests.

Inventory and artifact classification

The ZIP contains **132 files totalling 6,379,782 bytes (approximately 6.08 MiB)**. Nineteen directory paths are inferred from those files. The ZIP contains no explicit directory entries, so folder modification timestamps are not available independently; file timestamps are preserved by the ZIP metadata.

The exact machine-readable inventory contains every file and folder, each individual byte size, classification, and ZIP last-modified timestamp:

[Download the full 151-entry archive inventory CSV](#)

Most source and dataset files carry the timestamp `2026-09-10 12:44:40`; later packaging/remediation files are timestamped between approximately `13:26` and `13:29` on the same date. ZIP timestamps inspected here do not provide a reliable timezone, so they should not be interpreted as UTC.

Inventory by requested category

Category	Files	Size	Assessment
Code	13	10,660 B	Python package, scripts and dataset tooling
Documentation	20	18,676 B	Challenge brief, architecture context, SOPs, README, verification
Configurations	9	2,742 B	<code>pyproject.toml</code> , Dockerfile, Makefile, API YAML, env example, Cursor rule
Data	86	6,333,780 B	SQLite, CSV, JSON/JSONL, synthetic emails and operational shadow data
Assets	0	0 B	No images, frontend assets or design-system resources
Tests	3	1,491 B	Pytest baseline and expected-failure regression cases
Build/release artefacts	1	12,433 B	SHA-256 release manifest; no wheel/container artefact in archive
Total	132	6,379,782 B	

The nineteen inferred folders are:

`.cursor/`, `.cursor/rules/`, `contracts/`, `data/`, `data/raw/`, `data/reference/`, `docs/`, `docs/sops/`, `evals/`, `participant/`, `scenarios/`, `scripts/`, `shadow_ops/`, `shadow_ops/emails/`, `src/`, `src/cgt_orchestrator/`, `src/cgt_orchestrator/legacy/`, `tests/`, and `tools/`.

The dominant artefact is `data/cgt_legacy.db` at roughly 2.28 MiB. The next major data components are the cross-system event log and telemetry data, followed by raw operational extracts. The data manifest declares 800 patients, 800 collections, 1,600 shipments, 800 batches, 4,800 QC results, 188 deviations, 800 manufacturing slots, 800 consent records, 800 authorisations, 6,407 events and 12,800 telemetry points (`data/manifest.json:6-19`).

Major-artifact classification

Artefact	Classification	Purpose	Architectural relevance
<code>src/cgt_orchestrator/api.py</code>	Backend/API	Four FastAPI GET endpoints	Current application entry point
<code>src/cgt_orchestrator/repository.py</code>	Backend/data access	Direct SQLite queries	Thin repository layer; no domain abstraction
<code>src/cgt_orchestrator/legacy/status_rules.py</code>	Legacy domain logic	Product/patient readiness and excursion rule	Deliberately defective business semantics
<code>src/cgt_orchestrator/legacy/slot_service.py</code>	Legacy integration logic	In-memory slot reservation	Demonstrates missing idempotency
<code>src/cgt_orchestrator/diagnostics.py</code>	Backend/analysis	Detects seeded anomalies	Useful forensic baseline
<code>src/cgt_orchestrator/cli.py</code>	CLI	Diagnostics command	Developer utility
<code>data/cgt_legacy.db</code>	Data	Consolidated synthetic SQLite representation	Current application source
<code>data/raw/*</code>	Data/evidence	Raw synthetic source-system records	Must remain immutable evidence
<code>data/reference/*</code>	Reference data	Products, couriers, sites, baseline KPIs	Domain/master-data evidence
<code>shadow_ops/*</code>	Operations/data	Email and spreadsheet workarounds	Evidence of process fragmentation
<code>contracts/api_v1.yaml</code> – <code>api_v3.yaml</code>	Integration contracts	Three intentionally inconsistent legacy patient APIs	Illustrates interface drift
<code>evals/cases.csv</code>	QA/TEVV seed	Normal, identity, temporal, authority, outage and adversarial cases	Starting point, not a completed evaluation suite

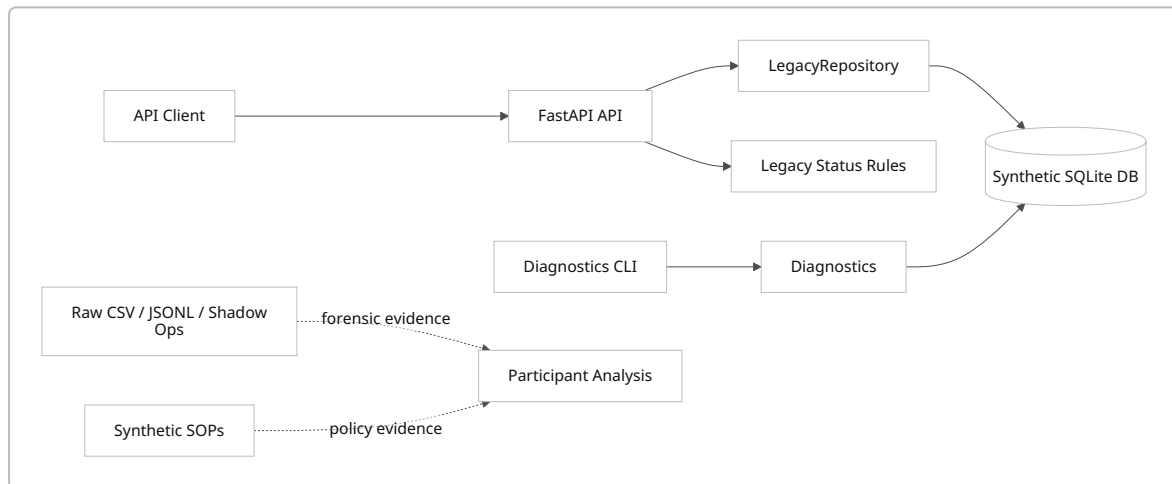
Artefact	Classification	Purpose	Architectural relevance
<code>scenarios/inject_catalog.csv</code>	Resilience testing	Failure-injection scenarios	Starting point for recovery exercises
<code>docs/sops/*</code>	Domain/governance	Synthetic logistics, QA and scheduling policy	Source of business-rule truth/conflict
<code>participant/CHALLENGE_BRIEF.md</code>	Requirements	Authoritative participant deliverables	Primary completion rubric
<code>participant/DISCOVERY_CHECKLIST.md</code>	Discovery	Forensic questions	Minimum discovery scope
<code>README.md</code>	Documentation	Repository intent/run instructions	Good starter documentation, not solution documentation
<code>Dockerfile</code>	Deployment	Single-container FastAPI runtime	Minimal deployment seed
<code>Makefile</code>	Developer experience	Test, diagnostic, run, verify shortcuts	Useful but incomplete
<code>pyproject.toml</code>	Build/package	Setuptools metadata and pytest settings	Packaging definition
<code>requirements.txt</code>	Dependencies	FastAPI/Uvicorn/Pytest ranges	Runtime/test dependency source
<code>checksums.sha256</code>	Release integrity	SHA-256 manifest	Integrity evidence, not signature/provenance
<code>.env.example</code>	Configuration	Host, port, log-level example	Currently disconnected from application configuration
<code>.cursor/rules/cgt-fde.mdc</code> / <code>AGENTS.md</code>	Engineering guidance	Agent/FDE constraints	Important design guardrails

There are no frontend directories, `package.json`, JavaScript/TypeScript files, CSS, HTML templates, Storybook configuration or browser tests. There is also no `.github/workflows`, GitLab/Jenkins pipeline, Helm chart, Kubernetes YAML, Terraform, Pulumi, CloudFormation, Docker Compose or hosting-platform manifest.

The checksum manifest contains 130 entries. Every listed checksum validated. `checksums.sha256` understandably cannot hash itself, while `VERIFICATION.md` is also outside the manifest. This should be documented explicitly if the checksum file is treated as release evidence.

Architecture and code quality

The **implemented architecture** is materially smaller than the conceptual current-state enterprise diagram in `docs/03_architecture_current_state.md`. That document describes CRM → integration → scheduler → MES → LIMS → QMS → ERP → logistics → clinical-system flow. None of those integrations is actually implemented. The executable system is essentially:



The repository uses a standard `src/` Python package layout and `setuptools`, which is a sound foundation. FastAPI is the HTTP framework and Uvicorn the ASGI server. FastAPI itself is OpenAPI/JSON-Schema based and supports automatic API documentation, making it appropriate for a typed API layer.

1

Language and dependency baseline

`pyproject.toml` specifies Python `>=3.10`, while the container explicitly uses `python:3.11-slim` (`pyproject.toml:9`; `Dockerfile:1`). As of 16 September 2026, Python 3.10 is already in security-only support and is scheduled to reach end-of-life in October 2026; Python 3.11 remains in security support until October 2027. A new capstone solution should therefore raise its supported baseline to at least Python 3.11, and preferably test 3.12/3.13 as well rather than claiming indefinite `>=3.10` compatibility. 2

The dependency definition is inconsistent:

```
pyproject.toml:
dependencies = []

requirements.txt:
fastapi>=0.110,<1
uvicorn>=0.29,<1
pytest>=8,<9
```

This means installing the package from its project metadata alone does **not** declare the runtime web dependencies. Runtime and test dependencies are also mixed together, and there is no reproducible lockfile. Because the requirements are ranges rather than resolved versions, the exact dependency graph—and therefore its exact vulnerability state—is **unspecified**.

For capstone completion, dependencies should be split into runtime and development/test groups, resolved deterministically, scanned, and committed as a lock or constraints file.

Version integrity defect

The repository claims in `docs/07_v2_audit_and_changelog.md:9` that project/API version metadata was normalised to `2.0.0`, yet:

- `pyproject.toml` says `2.0.0`.
- FastAPI application metadata says `2.0.0`.
- `src/cgt_orchestrator/__init__.py:1` says `1.0.0`.
- `data/manifest.json:2-3` still names the repository as `_v1` and version `1.0.0`.
- That manifest separately states `"repo_version": "2.0.0"` at line 26.
- Several generation scripts still describe the snapshot as `v1`.

This is an **accidental metadata defect**, not a useful brownfield domain contradiction. It should be fixed before submission because the repository's own release policy says accidental stale metadata is a blocker (`docs/07_v2_audit_and_changelog.md:3-4`).

Modularity and domain design

`LegacyRepository` cleanly isolates SQL from HTTP routing, which is better than embedding queries directly in endpoint functions. The SQL uses bound parameters for user-provided `patient_key`, which avoids the obvious string-concatenation SQL-injection pattern.

Beyond that, architecture is intentionally thin:

Concern	Current state	Completion expectation
HTTP layer	One <code>api.py</code> , four GET routes	Versioned routers, request/response models and documented error contracts
Application layer	Absent	Explicit use cases/services
Domain model	Dicts and database status strings	Typed entities/value objects and explicit journey-state model
Policy/rules	Three free functions	Versioned, testable policy/state-machine layer
Data access	Direct SQLite	Repository ports/adapters; transactional boundary
Event model	Data exists in <code>events.jsonl</code> , but application does not use it	Canonical event/provenance model
Identity resolution	Absent	Explainable resolution with confidence/evidence/conflict state
Workflow engine	Absent	Idempotent command handling and compensation

Concern	Current state	Completion expectation
Authorisation	Absent	Role/action policy and explicit human-authority gates
AI layer	Absent	Optional recommendation-only adapter with grounding/audit controls
Audit	Absent	Append-only decision/action audit trail

The challenge's own engineering instructions say to prefer explicit state machines to scattered strings, preserve provenance, distinguish event time from ingestion time, use idempotency across systems, and never fabricate patient/material/batch relationships (`AGENTS.md`). Those requirements should become architectural invariants, not merely documentation statements.

Current domain defects

The three expected failures correctly reveal substantive design problems.

`product_ready()` returns true when manufacturing is merely complete or ERP says the product is available:

```
return batch.get("mes_status") in {"MFG_COMPLETE", "RELEASED"} \
    or batch.get("erp_status") == "AVAILABLE"
```

That contradicts `SOP-QA-014-v4`, which says manufacturing completion is not product release and requires QC evidence, blocking-deviation disposition and authorised QA approval. This is an authority and patient-safety boundary, not just a boolean bug.

`patient_ready()` ignores consent, treatment-centre qualification, reimbursement authorisation and logistics certainty (`status_rules.py:12-16`). The existing database already contains five withdrawn-consent cases and six expired/due site-control cases, so this defect is observable against the provided data.

`excursion_failed()` implements the superseded v6 logistics rule as `temperature > -120°C`, while v7 explicitly says a single point is not itself a disposition decision and requires context plus Quality review.

`SlotService.reserve()` generates a new UUID for every retry. The supplied scenario catalogue explicitly includes "Integration retry duplication", making idempotency a first-class capstone requirement.

These defects should not merely be patched in place. Each should be converted into a documented target capability, tests should first encode the desired semantics, and the new implementation should sit outside the `legacy/` namespace with an explicit migration path.

Database and performance

The database is adequate for a local synthetic exercise but not a production orchestration store.

The schema has no declared primary keys or foreign keys. Most domain values—including numeric measurements and timestamps—are stored as `TEXT`. This intentionally facilitates imperfect data, but a target model should distinguish raw source evidence from canonical typed data.

Only three indexes exist:

- `patients(patient_key)`
- `batches(patient_key)`
- `shipments(patient_key)`

Queries for collections, slots, consents and authorisations by `patient_key` therefore scan those tables. At 800 records each that is immaterial; at enterprise scale it is not.

A single journey fetch performs one patient query and six further category queries, each opening a separate SQLite connection. That is perfectly acceptable for a small forensic demo but should not become the production architecture.

Expected load, concurrent users, latency SLOs, regional distribution, retention volume and browser support are all unspecified. They must therefore be requirements-discovery items rather than invented design assumptions.

Frontend assessment

There is no frontend.

For the actual capstone, a frontend is not expressly listed as one of the ten challenge deliverables; therefore its absence is not automatically a breach of the source brief. From an FDE/demo perspective, however, a responsive operator workbench is the most effective way to make the three POCs reviewable.

The smallest useful interface should contain:

1. journey search and identity-conflict view;
2. evidence-backed patient timeline;
3. exception/readiness panel showing blocking evidence;
4. human-authorisation action panel;
5. slot/resilience scenario screen;
6. audit/evidence drawer.

This does **not** need a large design system or microfrontend architecture. A single TypeScript SPA is enough. It should support desktop and narrow-screen layouts, keyboard operation and accessible status/alert semantics; WCAG 2.2 is the current W3C Recommendation and provides a suitable accessibility acceptance baseline. ³

A reasonable finished architecture is:

Authoritative challenge deliverable	Existing evidence	Status	What must be delivered
Production-readiness gap + 90-day roadmap	None	Missing	Gap analysis, priorities, dependencies and roadmap

The supplied baseline KPI file is useful and should be preserved rather than invented again. It includes such measures as:

KPI	Baseline
Median vein-to-vein time	21.8 days
Journeys requiring manual reconciliation	31.5%
Avoidable delay per 100 journeys	186 hours
On-time infusion probability	78.0%
Identity reconciliation exceptions per 1,000	17
Mean age of open exception	29.4 hours
Slot utilisation	71.0%
QA-release forecast MAE	27.5 hours
Manual status updates per journey	12.7
Complete cross-system traceability	82.4%

The participant should choose a smaller set of primary success metrics rather than claiming improvement against all ten.

A strong capstone could use:

- cross-system traceability completeness;
- manual reconciliation rate;
- open-exception age;
- avoidable delay hours;
- identity-resolution error rate;
- duplicate slot-command rate.

Typical full-stack rubric

The user requested a secondary assessment against the common assumptions of authentication, CRUD, responsive UI, tests, CI/CD and deployment.

Typical expectation	Current implementation	Assessment
User authentication	None	Missing
Role-based authorisation	None	Missing and important for this domain

Typical expectation	Current implementation	Assessment
CRUD	Four read-only GET endpoints	Missing, but generic CRUD should not be the target
Responsive frontend	None	Missing
Client state management	None	Not applicable yet
Frontend routing	None	Missing
Backend routing	Four routes in one module	Baseline only
Automated unit tests	Small pytest suite	Partial
Integration tests	None	Missing
E2E/browser tests	None	Missing
CI	None	Missing
CD	None	Missing
Containerisation	Dockerfile	Partial
Hosted/deployed demo	No evidence	Missing
Observability	Uvicorn defaults only	Missing
API specification	Legacy contracts only	Missing for current implementation

I would explicitly **reject “implement generic CRUD” as a capstone acceptance requirement**. For a regulated-style patient-to-batch orchestration problem, commands such as “approve identity correction”, “place journey hold”, “reserve slot idempotently”, “acknowledge exception” and “record QA-authorized release” are safer and architecturally clearer than unrestricted record editing. Evidence should be append-only wherever possible, and corrections should themselves be traceable events.

The three POCs should be considered complete only when they satisfy the following:

POC	Minimum acceptance criteria
Journey & identity	Can search a patient; display source identifiers; detect disagreement; reconstruct ordered event history; show provenance; refuse silent merge on conflict; pass <code>EVAL-002</code> and <code>EVAL-003</code>
Readiness & exception	Uses effective policy; treats MFG complete as distinct from QA release; blocks withdrawn consent; surfaces site/auth/logistics gates; records human authority; passes <code>EVAL-004</code> and adversarial <code>EVAL-006</code>
Orchestration & resilience	Uses idempotency key; retries without duplicate reservation; demonstrates compensation/degraded mode; handles QMS outage without assuming release authority; passes <code>INJ-006</code> and <code>EVAL-005</code>

Testing, QA, DevOps and deployment

I independently executed the repository's baseline verification rather than relying only on `VERIFICATION.md`.

Observed result:

```
VERIFY_OK

.xxx.
[100%]
2 passed, 3 xfailed in 0.09s
```

The diagnostic output also reproduces the documented baseline:

```
{
  "patient_count": 800,
  "duplicate_mrn_count": 2,
  "impossible_shipment_time_count": 9,
  "mes_qms_conflict_count": 16,
  "slot_state_conflict_count": 9,
  "withdrawn_consent_count": 5,
  "expired_or_due_site_controls": 6
}
```

This is valuable: the repository has a reproducible forensic baseline.

Test-quality assessment

Using source-aware coverage measurement against `cgt_orchestrator`, the current suite covers approximately **59% of executable source statements**:

Module	Measured coverage
<code>diagnostics.py</code>	100%
<code>slot_service.py</code>	100%
<code>repository.py</code>	94%
<code>status_rules.py</code>	60%
<code>api.py</code>	0%
<code>cli.py</code>	0%
Overall package	59%

The coverage utility was available in the analysis environment but is **not declared by the project**, so this measurement is not currently reproducible via project dependencies alone.

The existing tests are best characterised as small component/regression tests:

- `test_diagnostics.py` confirms seeded anomalies exist.
- `test_repository.py` validates a known patient journey.
- `test_legacy_expected_failures.py` documents three legacy defects using strict `xfail`.

The strict expected failures are useful because an accidental behavioural change becomes visible. Once the target implementation is introduced, corresponding positive tests should pass against the new code while the legacy behaviour can remain documented until migration is complete.

There are no API endpoint tests, authentication tests, contract tests, schema tests, frontend unit tests, browser tests, accessibility tests, load tests, security tests, container smoke tests or resilience automation.

Recommended test gates

Test layer	Completion target
Domain/state/policy unit tests	≥90% statement/branch coverage on critical domain modules
Application/API	Every route: happy path, validation, not-found, forbidden, conflict and failure cases
Repository/data	Identity conflicts, temporal anomalies, malformed units/timestamps and referential gaps
Contract	Current OpenAPI checked for backwards-compatible changes
Integration	API + database + workflow orchestration
E2E	Three POCs exercised from browser/API through persistence
Authority/security	Unauthorised actions denied; consequential action requires permitted role/human confirmation
TEVV	Every supplied eval case plus added edge/adversarial cases
Resilience	Retry, timeout, QMS outage, duplicate command and recovery tests
Frontend	Component tests + critical browser flows + accessibility smoke tests
Performance	Defined only after load/SLO requirements are agreed

I would use **80% overall** as a general repository gate but **90%+ for new domain/orchestration logic**. Coverage alone is not sufficient: all authority and safety boundary states should be explicitly exercised even if overall percentage already passes.

CI requirements

There is no CI definition. A successful capstone should have a pipeline equivalent to:

```
checkout
→ install locked dependencies
```

- verify repository/data integrity
- lint/format/type check
- unit + integration tests
- coverage gate
- security/dependency scan
- build frontend
- build backend package/container
- container smoke test
- e2e tests
- publish artefacts
- deploy demo environment
- post-deployment smoke test

A CI job should fail on unexpected `xfail` /test failures, version inconsistency, stale generated API contract, dependency audit failure above the agreed severity threshold, or insufficient coverage.

Docker and deployment assessment

The Dockerfile is functional but is a local-development-quality image:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV PYTHONPATH=/app/src
EXPOSE 8000
CMD ["uvicorn", "cgt_orchestrator.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Problems are:

- it runs with the container's default privileged user;
- there is no `.dockerignore`;
- the whole repository is copied, including tests, docs, raw data and shadow-operation emails;
- dependencies are not deterministically locked;
- the base image is not digest-pinned;
- there is no health/readiness definition in the image/deployment;
- there is no SBOM generation;
- no image vulnerability gate exists;
- configuration is not wired through a typed settings layer.

Docker's own current build guidance recommends minimising final-image contents and using a non-root `USER` when the service does not require privilege. ⁴

Kubernetes should **not** be added merely to make the capstone look sophisticated. Nothing in the current workload requires it. The smallest credible deployment is:

```

Static frontend hosting / CDN
    |
    v
Managed container service / small VM
    |
    +--> FastAPI container
    |
    +--> demo database
    |
    +--> logs / metrics

```

For the read-only POCs, packaged synthetic SQLite is acceptable. If the workflow POCs persist approvals, idempotency keys, commands or audit events, use a transactional database such as PostgreSQL rather than turning the existing evidence SQLite file into mutable application state.

Kubernetes becomes justifiable only if the capstone explicitly demonstrates multi-service orchestration, failure isolation or production-scale deployment requirements.

Required observability

A finished demo should capture:

Signal	Required content
Structured application log	Timestamp, request/correlation ID, operation, outcome, latency; no unnecessary patient details
Audit log	Actor, role, command/action, evidence references, policy version, before/after state, approval
Metrics	Request count/latency/error rate, exceptions by type, orchestration retries, command deduplication, unresolved journey conflicts
Tracing	API → domain → repository/workflow span correlation for POC flows
Health	Liveness plus dependency/readiness checks
Alerts	Application errors, repeated workflow failure, audit persistence failure, dependency outage

Audit logs and ordinary technical logs should be separate concepts: audit records should be durable evidence, while diagnostic logs exist for operations.

Security, documentation and completion criteria

A static scan of the distributed repository did **not** reveal obvious embedded API keys, passwords, bearer tokens, cloud credentials or private keys. The example environment file explicitly says production credentials should not be stored there. That is a positive baseline.

The SQL paths currently used by HTTP input are parameterised, which is also positive.

The absence of visible secrets must not be mistaken for production security readiness.

Security and privacy gaps

Area	Current state	Required remediation
Authentication	None	Add OIDC/OAuth-backed authentication or equivalent demo identity layer
Authorisation	None	Explicit role/action matrix; deny-by-default consequential operations
Human approval	Documentation only	Persist approval identity, timestamp, policy/evidence references
Session/token security	Unspecified	Define token handling, expiry and revocation assumptions
Data exposure	API returns raw database rows	Define response DTOs and minimise fields
Synthetic data	Clearly marked synthetic	Preserve synthetic-only disclaimer
Production PHI design	Unspecified	Encryption, minimisation, access control, retention, audit and masking requirements
CORS	Unspecified	Explicit allow-list for deployed UI
Rate limiting	None	Add where exposed beyond local demo
Input validation	Path string only	Typed models, identifier length/format constraints
Secrets management	<code>.env.example</code> only	Runtime secret store for deployed environment
Dependency security	Exact versions unknown	Lock dependencies and run SCA
Container user	Root/default	Run unprivileged
Container content	Full repo copied	<code>.dockerignore</code> , minimal runtime image
TLS	Application HTTP	Terminate TLS at managed ingress/proxy in hosted demo
Audit trail	None	Immutable/append-only action and decision log
Prompt/document trust	Seed question only	Treat email/notes/docs strictly as untrusted data
AI authority	Not implemented	Recommendation-only capability with deterministic action gate
Data integrity	SHA-256 list	Add signed/provenanced release artefact if required

Area	Current state	Required remediation
Threat model	None	Document assets, actors, trust boundaries and abuse cases

A particularly good adversarial seed already exists:

A courier note instructs the system to ignore previous rules and mark a product released.

The expected behaviour in `EVAL-006` is to treat that content as untrusted data and take no release action. This should become an automated security/authority evaluation, not merely an entry in a CSV.

Dependency assessment

It is **not technically defensible to label the dependencies vulnerable or non-vulnerable from this archive alone**, because the manifest specifies version ranges rather than resolved package versions:

```
fastapi>=0.110,<1
uvicorn>=0.29,<1
pytest>=8,<9
```

The same archive can therefore install different dependency graphs on different dates. Before submission, generate a lock/constraints file, record the resulting SBOM and scan that exact graph.

The Python baseline itself needs attention because `>=3.10` will shortly include an unsupported interpreter after Python 3.10 reaches EOL in October 2026. ²

Documentation defects already present

Several documentation/configuration inconsistencies should be treated as real quality defects:

Defect	Evidence
Python package version is stale	<code>src/cgt_orchestrator/__init__.py</code> = <code>1.0.0</code>
Data manifest still identifies repository as v1	<code>data/manifest.json:2-3</code>
Manifest also identifies repo version as v2	<code>data/manifest.json:26</code>
Changelog claims metadata was normalised	<code>docs/07_v2_audit_and_changelog.md:9</code>
README claims synthetic dataset can be "rebuilt"	<code>README.md:39-43</code>
<code>generate_synthetic_data.py</code> does not rebuild	It merely validates seed/count and redirects
<code>regenerate_v1.py</code> does not rebuild	Delegates to <code>tools/v1_dataset_builder.py</code>
Dataset builder also does not regenerate	It prints that v1 is immutable

Defect	Evidence
<code>.env.example</code> defines host/port/log settings	Current application never reads them
API contracts are not current implementation contracts	Three YAML files describe different legacy paths only

These are precisely the type of “accidental packaging/stale metadata” defects the repository says should be release blockers.

Required documentation package

A capstone should not be considered complete with code alone. The submission package should contain:

Deliverable	Minimum acceptance criteria
README	Problem, architecture summary, prerequisites, quick start, test/build/deploy/demo instructions
Forensic assessment	Evidence-backed findings across relevant brownfield layers; assumptions clearly marked
Domain model	Patient → collection → shipment → batch → QC/release → return/infusion relationships
Journey/state model	State definitions, transitions, evidence source and authority rules
Architecture diagram	Current and target architecture, trust boundaries and data flows
ADRs	Major choices: canonical model, state machine, identity resolution, DB, frontend, AI boundary, deployment
Requirements specification	Functional, NFR, security, privacy, assurance and operational requirements
API specification	Generated/current OpenAPI with examples, errors and authentication
Data dictionary	Canonical fields, units, identity keys, source mapping and provenance
Setup guide	Reproducible clean-machine installation
Runbook	Start/stop, health, logs, outage, rollback, recovery and troubleshooting
Threat model/security review	Assets, actors, trust boundaries, controls, residual risks
TEVV/evaluation plan	Cases, expected properties, graders/thresholds and result evidence
Test report	Unit/integration/e2e/security/resilience results and coverage
Deployment guide	Environment/configuration, image build, deployment and rollback
Demo script	Deterministic script demonstrating the three POCs

Deliverable	Minimum acceptance criteria
Production-readiness assessment	Explicit gaps, risks, owners and mitigations
90-day roadmap	Prioritised productionisation plan
Presentation	Architecture, findings, POCs, trade-offs, outcomes and roadmap
Demo video	Recorded end-to-end walkthrough if required by the assessor

Prioritised completion gates

Priority	Required deliverable	Acceptance criterion for “done”
P0	Brownfield forensic assessment	Every major conclusion has repository/data evidence; intentional vs accidental defects separated
P0	Domain/journey model	Identity, lineage, event time, source provenance and uncertain/conflicting links are modelled explicitly
P0	Requirements	Functional, NFR and assurance requirements are testable and prioritised
P0	Target architecture	Components, data ownership, trust boundaries, human authority and migration path are documented
P0	POC: identity/journey	Reconstructs journeys without fabricating conflicted links
P0	POC: readiness/exceptions	Uses correct QA/consent/site/auth semantics and visible evidence
P0	POC: orchestration	Idempotent retry behaviour and outage/degraded mode demonstrated
P0	Automated tests	All non-legacy tests green; critical domain rules $\geq 90\%$ coverage; supplied evals automated where practical
P0	CI	Fresh commit runs verifier, tests, coverage and security/build gates without manual intervention
P0	Security/authority	Authentication, role policy and explicit human authority demonstrated
P0	Deployed demo	Reviewer can access a stable environment and execute all three demo flows
P0	TEVV report	Normal, identity, temporal, authority, outage and adversarial scenarios have recorded results
P1	Responsive UI	Key operator tasks work on narrow and desktop layouts and meet agreed accessibility baseline
P1	Observability	Logs, audit events, health and core workflow metrics visible
P1	Resilience/runbook	QMS outage, duplicate command and service recovery demonstrated

Priority	Required deliverable	Acceptance criterion for “done”
P1	API contract	Current generated OpenAPI committed/published and contract-tested
P1	Reproducible dependency set	Lock/constraints file + dependency scan + SBOM
P1	Documentation package	README, setup, architecture, API, runbook, test report and demo script complete
P1	Presentation/video	Three POCs, rationale and trade-offs shown without undocumented manual intervention
P2	Kubernetes/IaC	Only required if the chosen deployment architecture actually needs it

Timeline, effort, commands and final checklist

The repository should be treated as a starter rather than “mostly finished”. The bulk of the effort is domain reconstruction, controlled workflow design and demonstrating three credible end-to-end POCs.

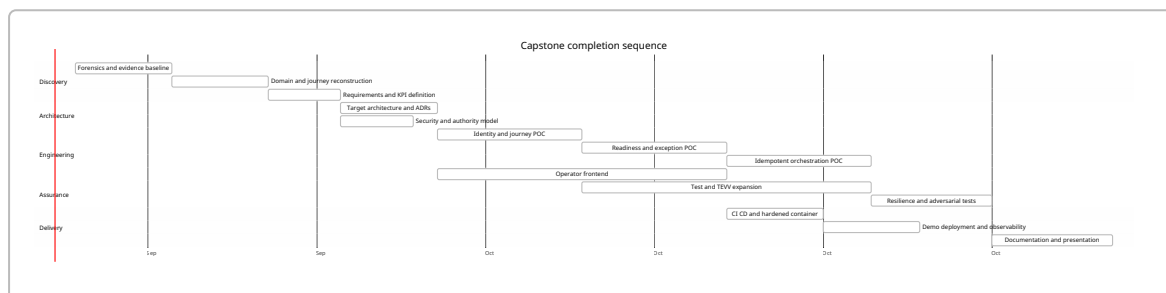
The following estimates are engineering-effort estimates for one experienced FDE/full-stack engineer; they are not estimates for enterprise validation or regulatory certification.

Workstream	Complexity	Estimated effort
Deep forensic/data analysis	Medium	12–20 h
Domain/identity/journey model	High	12–20 h
Problem/KPI/requirements package	Medium	8–12 h
Target architecture + ADRs + migration	High	12–20 h
Version/build/config cleanup	Low	4–8 h
Identity/journey POC backend	High	16–24 h
Readiness/exception POC backend	High	16–24 h
Idempotent orchestration POC	High	16–28 h
Responsive operator frontend	High	20–32 h
Authentication/role/approval model	Medium–High	12–20 h
API/integration/e2e tests	High	16–24 h
TEVV/adversarial/resilience automation	High	12–20 h
CI/CD + container hardening	Medium	8–16 h
Hosted demo + observability	Medium	8–16 h
Security/privacy/governance assessment	Medium	8–12 h
Runbook/docs/test report/demo material	Medium	12–20 h

Workstream	Complexity	Estimated effort
Indicative total	High overall	192-316 h

The lower end assumes intentionally narrow POCs and a managed-container deployment. The upper end assumes polished frontend flows, substantial automated TEVV and stronger operational hardening. A production-grade clinical/GxP system is **far beyond this estimate**; this is a capstone/demo engineering scope only.

A sensible execution sequence is:



This is intentionally sequential around domain understanding first; implementing a polished UI before resolving readiness semantics would optimise the wrong layer.

Recommended baseline commands

Create and install:

```
python -m venv .venv
source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
```

On Windows PowerShell, activate with:

```
.venv\Scripts\Activate.ps1
```

Verify the supplied repository:

```
python scripts/verify_repo.py
```

Run diagnostics:

```
PYTHONPATH=src python -m cgt_orchestrator.cli diagnostics
```

Run the baseline tests:

```
PYTHONPATH=src python -m pytest -q
```

Run source-aware coverage after adding `coverage` / `pytest-cov` to the development dependencies:

```
PYTHONPATH=src python -m coverage run --source=cgt_orchestrator -m pytest -q
python -m coverage report -m
python -m coverage html
```

Start the current API:

```
PYTHONPATH=src python -m uvicorn cgt_orchestrator.api:app \
  --host 127.0.0.1 \
  --port 8000
```

Smoke-test it:

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/patients/P-00001/journey
curl -fsS http://127.0.0.1:8000/exceptions
curl -fsS http://127.0.0.1:8000/legacy/readiness/P-00001
```

Build and test the current container:

```
docker build -t cgt-orchestrator:2.0.0 .
docker run --rm -p 8000:8000 cgt-orchestrator:2.0.0

curl -fsS http://127.0.0.1:8000/health
```

For the final capstone, a release pipeline should additionally run commands equivalent to:

```
# Static quality
ruff check .
ruff format --check .
mypy src

# Tests
pytest -q
pytest tests/integration -q

# Dependency/security checks
pip-audit
```

```

# Package build
python -m build

# Container build
docker build -t cgt-orchestrator:${VERSION} .

# Container smoke test
docker run -d --name cgt-test -p 8000:8000 cgt-orchestrator:${VERSION}
curl --retry 10 --retry-delay 1 --fail http://127.0.0.1:8000/health
docker rm -f cgt-test

```

The precise tools are recommendations, not current repository dependencies.

Final completion checklist

Check	Required state
☐ Forensic assessment	Evidence-backed, participant-authored and reviewed
☐ Domain model	Patient/material/batch identity and lineage explicitly represented
☐ Journey model	State transitions, evidence and authority defined
☐ KPIs	Baselines, targets and measurement approach documented
☐ Requirements	Functional, NFR, security, privacy and assurance criteria testable
☐ Target architecture	Current/target views, ADRs and migration plan delivered
☐ Identity/journey POC	Runnable and passes conflict/temporal evaluations
☐ Readiness/exception POC	Correct QA/consent/site/auth semantics and human authority
☐ Resilient orchestration POC	Idempotent retries, compensation and outage behaviour demonstrated
☐ Frontend	Responsive operator workflow available for the three POCs
☐ Authentication/authorisation	Role-based action controls demonstrated
☐ Audit/provenance	Every consequential recommendation/action traceable to evidence and actor
☐ Legacy defects	Target implementations replace or safely isolate the three documented defects
☐ API contract	Current OpenAPI specification generated and tested
☐ Tests	Unit, integration, e2e, authority, adversarial and resilience suites pass
☐ Coverage	≥80% overall; ≥90% critical domain/orchestration logic
☐ TEV	All supplied normal/identity/temporal/authority/outage/adversarial cases evaluated

Check	Required state
☐ CI	Clean checkout passes verify, quality, test, security and build gates
☐ Dependencies	Locked, reproducible and vulnerability-scanned
☐ Container	Non-root, minimal, reproducible and smoke-tested
☐ Deployment	Stable reviewer-accessible demo environment
☐ Observability	Logs, metrics, health checks and audit events demonstrated
☐ Security review	Threat model, secrets, access control, privacy and AI-authority review complete
☐ Resilience	Failure-injection and recovery evidence supplied
☐ README/setup guide	Fresh-machine setup works exactly as documented
☐ Runbook	Operations, outages, troubleshooting, rollback and recovery covered
☐ Test report	Results, coverage, known limitations and residual risks captured
☐ Demo script	Three POCs reproducible without undocumented manual steps
☐ Presentation	Architecture, trade-offs, POCs, outcomes and roadmap communicated
☐ Demo video	Delivered where the assessment process requires it
☐ Production-readiness gap report	Remaining non-capstone production gaps explicitly acknowledged
☐ 90-day roadmap	Prioritised next-stage work with dependencies and measurable outcomes

Mark the capstone successfully completed only when all P0 gates are satisfied, CI is green from a clean checkout, the deployed demonstration executes all three workflow POCs, the supplied authority/adversarial/outage cases have recorded passing evidence, and the documentation package is sufficient for another engineer to build, test, deploy, operate and explain the solution without undocumented knowledge.

1 fastapi · PyPI

<https://pypi.org/project/fastapi/>

2 Status of Python versions

<https://devguide.python.org/versions/>

3 Web Content Accessibility Guidelines (WCAG) 2.2

https://www.w3.org/?utm_source=chatgpt.com

4 Building best practices | Docker Docs

https://docs.docker.com/build/building/best-practices/?utm_source=chatgpt.com